

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Sementara Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikrokontroler

Rozaq Nafi'ul Hafidz - 5024241072

2026

1 Pendahuluan

1.1 Latar Belakang

Mikrokontroler merupakan komponen utama sistem tertanam yang menghubungkan pengolahan data dengan kondisi fisik di lingkungan. Perangkat ini digunakan pada pengendali robot, otomasi industri, peralatan rumah tangga, dan perangkat *Internet of Things* (IoT). Mikrokontroler menerima masukan dari tombol atau sensor, menjalankan algoritma, kemudian menghasilkan keluaran untuk mengendalikan perangkat lain. Oleh karena itu, pemahaman pemrograman dan antarmuka masukan-keluaran menjadi dasar penting sebelum merancang sistem tertanam yang lebih kompleks.

Permasalahan awal pengembangan sistem tertanam tidak hanya terletak pada penulisan kode. Pemilihan board, framework, dan metode pengunggahan program menentukan apakah kode dapat dibangun dan dijalankan pada perangkat yang digunakan. STM32 Blackpill dan ESP32 DevKit memiliki karakteristik perangkat keras serta antarmuka pemrograman yang berbeda. Penggunaan Arduino pada STM32 dan ESP-IDF pada ESP32 memperkenalkan pengaruh tingkat abstraksi perangkat lunak terhadap cara mengakses perangkat keras.

Pada tingkat rangkaian, masukan digital yang dibiarkan mengambang dapat terbaca tidak menentu. Tombol mekanik juga dapat menghasilkan beberapa transisi meskipun hanya ditekan sekali akibat fenomena *bouncing*. Kedua kondisi ini dapat menyebabkan kesalahan penghitungan atau perubahan keadaan sistem yang tidak diinginkan. Resistor pull-up/pull-down dan teknik *debouncing* diperlukan untuk menghasilkan pembacaan tombol yang andal. Selain masukan digital, banyak besaran fisik berubah secara kontinu, sehingga diperlukan *Analog-to-Digital Converter* (ADC) untuk mengolahnya.

Praktikum Modul 1: Pengantar Mikrokontroler dilaksanakan agar praktikan mampu menyiapkan VSCode dan PlatformIO, menjalankan program Blink, membaca tombol secara stabil, serta membaca sensor LDR melalui ADC. Rangkaian percobaan ini menghubungkan konsep pemrograman dengan perilaku listrik yang nyata. Penguasaan konsep tersebut menjadi bekal untuk mengembangkan antarmuka pengguna, pemantauan cahaya, dan sistem kendali berbasis sensor pada aplikasi robotika maupun IoT.

1.2 Dasar Teori

1.2.1 Mikrokontroler dan sistem tertanam

Mikrokontroler adalah rangkaian terpadu yang menggabungkan unit pemroses (CPU), memori, serta periferal masukan-keluaran pada satu chip. Memori Flash menyimpan program, sedangkan RAM menyimpan data sementara selama program berjalan. Periferal seperti GPIO, timer, UART, dan ADC memungkinkan CPU berinteraksi dengan perangkat luar. Sistem tertanam memanfaatkan komponen tersebut untuk menjalankan fungsi tertentu, misalnya membaca sensor dan mengendalikan indikator berdasarkan hasil pembacaan.

Mikroprosesor merujuk terutama pada unit pemroses. Mikrokontroler memadukan CPU, memori, dan periferal untuk pengendalian. Board pengembangan menambahkan catu daya, konektor, dan komponen pendukung agar chip lebih mudah digunakan. Karena itu, fitur chip perlu dibedakan dari fitur tambahan pada board, termasuk chip USB-to-Serial.

1.2.2 VSCode, PlatformIO, dan framework

Visual Studio Code (VSCode) merupakan editor kode yang dapat diperluas dengan ekstensi. PlatformIO IDE menyediakan pengelolaan proyek sistem tertanam, pengunduhan toolchain, kompilasi, pengunggahan firmware, dan Serial Monitor. *Toolchain* mencakup perangkat untuk mengubah kode sumber menjadi firmware, sedangkan *framework* menyediakan pustaka dan struktur program untuk mengakses fitur mikrokontroler.

Konfigurasi PlatformIO disimpan dalam `platformio.ini`. Parameter `platform` menentukan platform pengembangan, `board` menentukan target perangkat, `framework` memilih kerangka perangkat lunak, dan `upload_protocol` memilih protokol pengunggahan. Kode utama berada di folder `src/`; folder `include/` memuat header dan `lib/` menyimpan pustaka lokal. Konfigurasi yang benar diperlukan agar pemetaan pin, ukuran memori, serta proses pembangunan firmware sesuai perangkat.

1.2.3 STM32 Blackpill, Arduino, dan SWD

Board Blackpill pada modul menggunakan STM32F401CCU6 dengan CPU ARM Cortex-M4 32-bit, frekuensi maksimum 84 MHz, Flash 256 KB, dan RAM 64 KB. LED bawaan terhubung ke PC13 dan aktif LOW: keluaran LOW menyalakan LED, sedangkan HIGH mematikannya. Sifat aktif LOW ditentukan oleh rangkaian LED, sehingga tidak semua indikator menyala ketika pin diberi HIGH.

Framework Arduino untuk STM32, dikenal sebagai STM32duino, menyediakan fungsi seperti `pinMode()`, `digitalWrite()`, dan `digitalRead()`. Fungsi `setup()` menjalankan inisialisasi sekali, kemudian `loop()` dieksekusi berulang. Lapisan abstraksi ini mempermudah pemrograman tanpa mengharuskan pengguna mengatur register periferal secara langsung. Pada praktikum, ST-Link eksternal mengunggah firmware melalui *Serial Wire Debug* (SWD), menggunakan jalur SWDIO, SWCLK, ground bersama, dan sambungan/referensi 3,3 V sesuai perangkat.

1.2.4 ESP32 DevKit, ESP-IDF, dan komunikasi serial

ESP32 klasik pada ESP32 DevKit v1 merupakan mikrokontroler 32-bit berbasis Xtensa LX6 dengan dua inti pemroses, Wi-Fi, dan Bluetooth. Board DevKit yang dimaksud modul menyediakan chip USB-to-Serial, misalnya CP2102 atau CH340, untuk menjembatani USB komputer dengan UART ESP32. Firmware diterima bootloader ESP32 saat perangkat berada dalam mode pengunduhan. Jembatan USB-to-Serial merupakan komponen board, bukan fitur USB native pada chip ESP32 klasik.

ESP-IDF (*Espressif IoT Development Framework*) adalah framework resmi Espressif. Program pengguna dimulai melalui `app_main()` dan dapat memanfaatkan FreeRTOS untuk menjalankan task, antrean data, dan sinkronisasi. GPIO diatur dengan fungsi seperti `gpio_set_direction()` dan `gpio_set_level()`. Pemanggilan `vTaskDelay(pdMS_TO_TICKS(500))` menangguhkan task pemanggil selama interval sekitar 500 ms sesuai resolusi tick, sehingga scheduler dapat menjalankan task lain yang siap.

1.2.5 GPIO, pull-up, dan pull-down

General-Purpose Input/Output (GPIO) dapat dikonfigurasi sebagai keluaran atau masukan digital sesuai kemampuan pin. Keluaran mengendalikan level logika, sedangkan masukan membaca tegangan sebagai HIGH atau LOW berdasarkan ambang pada spesifikasi perangkat. Pin masukan dengan im-

pedansi tinggi yang tidak terhubung ke level tegangan tertentu berada pada kondisi *floating*, sehingga mudah terpengaruh derau.

Resistor pull-up menghubungkan masukan ke 3,3 V; tombol menghubungkannya ke ground saat ditekan. Resistor pull-down menghubungkan masukan ke ground; tombol menghubungkannya ke 3,3 V saat ditekan. Resistor mempertahankan kondisi awal dan membatasi arus ketika tombol ditutup. Untuk resistor $10\,\mathrm{k\Omega}$ pada 3,3 V, arus ideal saat tombol ditekan adalah

$$I = \frac{V}{R} = \frac{3{,}3}{10\,000} = 0{,}33\,\mathrm{mA}.$$

Pada ESP32 dengan Arduino, resistor internal dipilih melalui `INPUT_PULLUP` atau `INPUT_PULLDOWN`. Namun, GPIO34–GPIO39 pada ESP32 klasik merupakan masukan saja dan tidak memiliki resistor pull internal; pembacaan tombol pada pin tersebut memerlukan resistor eksternal.

1.2.6 Bouncing dan debouncing

Kontak tombol mekanik dapat memantul dan bergetar ketika bersentuhan atau terlepas. Akibatnya, satu aksi pengguna menghasilkan beberapa transisi logika dalam waktu singkat. Program yang langsung menghitung setiap tepi sinyal dapat mencatat lebih dari satu kejadian untuk satu tekanan.

Debouncing menyaring perubahan sementara tersebut. Pada metode perangkat lunak berbasis waktu, setiap perubahan pembacaan mentah memperbarui waktu acuan. Keadaan stabil baru diterima setelah pembacaan tidak berubah selama interval yang dipilih, misalnya 20–50 ms. Kejadian tekan dihitung hanya ketika keadaan stabil berpindah dari lepas ke tekan. Interval terlalu pendek dapat meloloskan pantulan, sedangkan interval terlalu panjang dapat mengabaikan tekanan cepat. Metode perangkat keras menggunakan penyaring RC, dapat disertai pemicu Schmitt, untuk meredam perubahan singkat dan membentuk transisi digital yang tegas.

1.2.7 ADC dan pembagi tegangan LDR

ADC mengubah tegangan analog menjadi kode digital melalui pencuplikan, kuantisasi, dan pengodean. ADC beresolusi N bit memiliki 2^N kemungkinan kode, dari 0 hingga $2^N - 1$. Untuk rentang masukan ideal $V_{\min}$ sampai $V_{\max}$, ukuran langkah kuantisasi adalah

$$\Delta V = \frac{V_{\max} - V_{\min}}{2^N}.$$

Pada model ideal 12-bit dengan rentang 0–3,3 V, terdapat 4096 kode dan langkah sekitar $0{,}806\,\mathrm{mV}$. Modul menggunakan pendekatan konversi kode ke tegangan dengan penskalaan titik ujung:

$$V_{\mathrm{perkiraan}} = \frac{D}{4095} \times 3{,}3\,\mathrm{V}, \qquad D \in \{0, 1, \ldots, 4095\}.$$

Persamaan tersebut merupakan model sederhana. Pada ADC ESP32, rentang masukan yang dapat diukur dan ketepatan hasil dipengaruhi attenuasi, karakteristik chip, kalibrasi, serta derau. Tegangan 3,3 V catu board tidak berarti seluruh rentang 0–3,3 V otomatis terukur secara linear pada setiap konfigurasi ADC. GPIO34 digunakan pada modul karena mendukung ADC1 dan merupakan pin masukan. ADC1 juga menghindari konflik pemakaian ADC2 dengan Wi-Fi pada ESP32 klasik.

LDR (*Light Dependent Resistor*) adalah resistor yang hambatannya umumnya menurun ketika intensitas cahaya meningkat. Pada rangkaian modul, LDR dipasang antara 3,3 V dan titik ukur, se-

dangkan resistor tetap $R = 10\,\mathrm{k}\Omega$ dipasang antara titik ukur dan ground. Tegangan keluarannya mengikuti

$$V_o = V_{\mathrm{in}} \frac{R}{R_{\mathrm{LDR}} + R}.$$

Ketika cahaya bertambah, hambatan LDR menurun sehingga V_o meningkat dan kode ADC cenderung membesar selama masih berada dalam rentang ukur. Pembacaan ini menunjukkan perubahan relatif pencahayaan; konversi menjadi satuan lux membutuhkan karakterisasi dan kalibrasi sensor.

2 Tugas Pendahuluan

1. **Jelaskan perbedaan mendasar framework Arduino dan ESP-IDF, termasuk kelebihan serta kekurangannya.**

   Perbedaan Framework Arduino dan ESP-IDF

   Perbedaan mendasar: Arduino menyederhanakan pemrograman, sedangkan ESP-IDF memberikan kontrol lebih rinci atas fitur dan sistem ESP32.

Aspek	**Arduino**	**ESP-IDF**
Pendekatan	API sederhana, seperti `digitalWrite()`.	API terperinci untuk konfigurasi perangkat dan sistem.
Struktur program	Umumnya menggunakan `setup()` dan `loop()`.	Menggunakan `app_main()`, task, dan komponen.
Kemudahan belajar	Mudah dipelajari pemula.	Memerlukan pemahaman sistem embedded dan FreeRTOS.
Kelebihan	Pengembangan prototipe cepat; banyak pustaka dan contoh.	Kontrol mendalam atas multitasking, memori, daya, dan konfigurasi sistem.
Kekurangan	Abstraksi dan konfigurasi bawaan dapat membatasi optimasi.	Penyiapan dan kode lebih kompleks; waktu belajar lebih lama.
Penggunaan	Praktikum, proyek sederhana, dan prototipe.	Sistem kompleks dan produk dengan kebutuhan teknis khusus.

   Hubungan keduanya: Arduino untuk ESP32 dibangun di atas ESP-IDF dan dapat digunakan sebagai komponen dalam proyek ESP-IDF. Arduino juga dapat mengakses kemampuan ESP-IDF dan FreeRTOS.

2. **Mengapa STM32 Blackpill menggunakan ST-Link eksternal, sedangkan ESP32 DevKit dapat diprogram melalui USB?**

   Penyebab utamanya adalah perbedaan rangkaian pada board atau perbedaan arsitektur antarmuka dan chip konverter bawaan.

   - **STM32 Blackpill:** Tidak memiliki ST-Link bawaan. Karena itu, pemrograman dan debugging melalui antarmuka SWD memerlukan ST-Link eksternal.

- **ESP32 DevKit:** Umumnya memiliki chip USB-to-UART bawaan, seperti CP2102 atau CH340. Chip ini menghubungkan komputer dengan bootloader internal ESP32 sehingga program dapat diunggah melalui kabel USB.
- **Catatan:** Blackpill berbasis STM32F401/F411 juga dapat diprogram melalui USB menggunakan bootloader DFU internal dengan mengatur BOOT0 dan melakukan reset. Jadi, ST-Link bukan satu-satunya metode pemrograman, tetapi juga menyediakan kemampuan debugging.

Alasan Teknis

- **STM32 Blackpill:** Mikrokontroler STM32 (misal: STM32F401/F411) mengekspos pin SWD (*Clock* dan *Data*) untuk debugging tingkat rendah dan pemrograman. Board murah sengaja menghilangkan chip USB-to-Serial onboard untuk menekan biaya ukuran, sehingga memerlukan programmer eksternal (ST-Link) atau adaptor USB-TTL terpisah.
- **ESP32 DevKit:** Chip ESP32 memiliki ROM bootloader internal yang mendukung pemrograman via UART serial. Board DevKit menyertakan chip jembatan USB-to-UART (seperti CP2102 atau CH340) serta sirkuit transistor otomatis yang mengatur pin `GPIO0` dan `EN` saat tombol upload ditekan dari komputer.

3. **Jelaskan kondisi HIGH/LOW pada pull-up dan pull-down ketika tombol dilepas maupun ditekan.**

   Tabel berikut mengasumsikan tombol pull-up menghubungkan GPIO ke ground, sedangkan tombol pull-down menghubungkan GPIO ke 3,3 V.

Konfigurasi	Tidak ditekan	Ditekan
Pull-up	HIGH	LOW
Pull-down	LOW	HIGH

   Pada pull-up, resistor menarik masukan ke 3,3 V ketika tombol terbuka. Ketika ditekan, tombol membuat jalur berhambatan rendah ke ground sehingga pin terbaca LOW; konfigurasi ini aktif LOW. Pada pull-down, resistor menarik masukan ke ground ketika tombol terbuka. Penekanan tombol menghubungkan pin ke 3,3 V sehingga terbaca HIGH; konfigurasi ini aktif HIGH.

   Prinsip tersebut sama untuk resistor internal dan eksternal. Penggunaan resistor mencegah masukan floating serta membatasi arus saat tombol ditekan. Untuk resistor internal, kemampuan pin perlu diperiksa terlebih dahulu; GPIO34–GPIO39 ESP32 klasik tidak menyediakan pull internal.

4. **Jelaskan penyebab bouncing dan minimal dua cara mengatasinya. Bouncing** terjadi ketika kontak mekanis tombol memantul saat ditekan atau dilepas, sehingga satu aksi menghasilkan beberapa perubahan logika singkat dan dapat terbaca sebagai beberapa penekanan.

   Cara mengatasinya:

   (a) **Debouncing perangkat lunak:** Terima perubahan status hanya setelah input stabil selama waktu tertentu, misalnya 10–50 ms, sesuai karakteristik tombol.

(b) **Debouncing perangkat keras:** Gunakan rangkaian resistor–kapasitor (RC) untuk menyaring pulsa singkat, lalu Schmitt trigger untuk menghasilkan transisi logika yang tegas.

5. **Bagaimana ADC mengubah sinyal analog menjadi digital, dan bagaimana resolusi bit memengaruhi pembacaan?**

   ADC (Analog-to-Digital Converter) mengubah tegangan analog menjadi angka digital melalui tiga tahap:

   (a) **Sampling:** mengambil nilai tegangan pada waktu tertentu.

   (b) **Kuantisasi:** memetakan tegangan ke tingkat diskrit terdekat.

   (c) **Pengodean:** menyatakan hasil sebagai bilangan biner.

   ADC beresolusi n bit memiliki 2^n tingkat, dengan kode 0 hingga $2^n - 1$. Untuk rentang input 0 sampai V_{ref}, ukuran satu langkah ideal adalah:

$$\Delta V = \frac{V_{\text{ref}}}{2^n}.$$

   Contoh untuk $V_{\text{ref}} = 3{,}3\,\text{V}$:

Resolusi	Jumlah tingkat	ΔV
8 bit	256	$12{,}89\,\text{mV}$
12 bit	4096	$0{,}806\,\text{mV}$

   Semakin tinggi resolusi bit, semakin kecil perubahan tegangan yang dapat dibedakan. Namun, akurasi tetap dipengaruhi noise, kualitas tegangan referensi, dan kalibrasi.

6. **Set-up VSCode, PlatformIO untuk ESP-IDF dan STM32, serta ekstensi Serial Plotter; sertakan tangkapan layar laptop.**

   a. Instal VSCode, kemudian pasang ekstensi *PlatformIO IDE* melalui menu Extensions. Tunggu pemasangan PlatformIO Core selesai dan pastikan PlatformIO Home dapat dibuka.

   b. Buat satu proyek Modul 1 sesuai arahan modul. Untuk STM32 Blackpill, gunakan konfigurasi `platformio.ini` berikut dan kode Blink STM32 pada `src/main.cpp`.

```
[env:blackpill_f401cc]
platform = ststm32
board = blackpill_f401cc
framework = arduino
upload_protocol = stlink
debug_tool = stlink
```

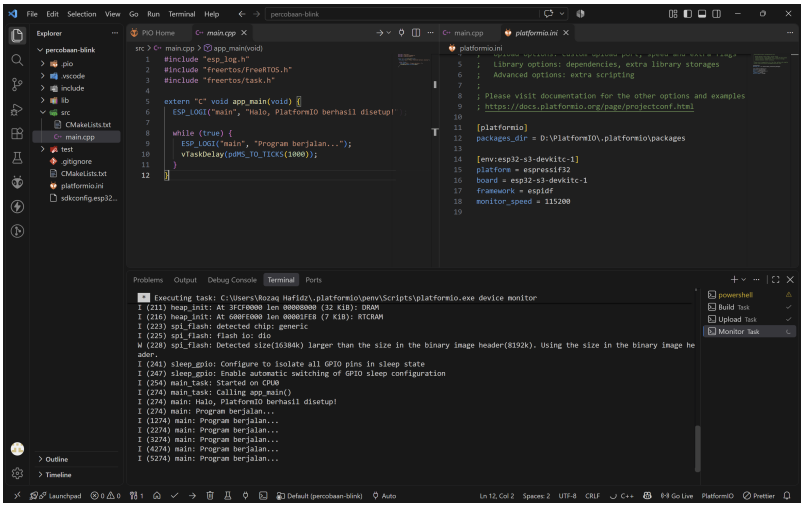
   Jalankan Build dan periksa hasilnya. Pengunggahan membutuhkan ST-Link beserta sambungan SWD yang benar. STM32 merupakan keluarga platform/board; framework yang digunakan pada modul adalah Arduino/STM32duino.

   c. Untuk percobaan ESP32 dengan ESP-IDF, ganti konfigurasi proyek yang sama menjadi berikut. Gunakan kode ESP-IDF pada `src/main.c` sesuai modul dan singkirkan sumber Blink STM32 yang tidak lagi dipakai.

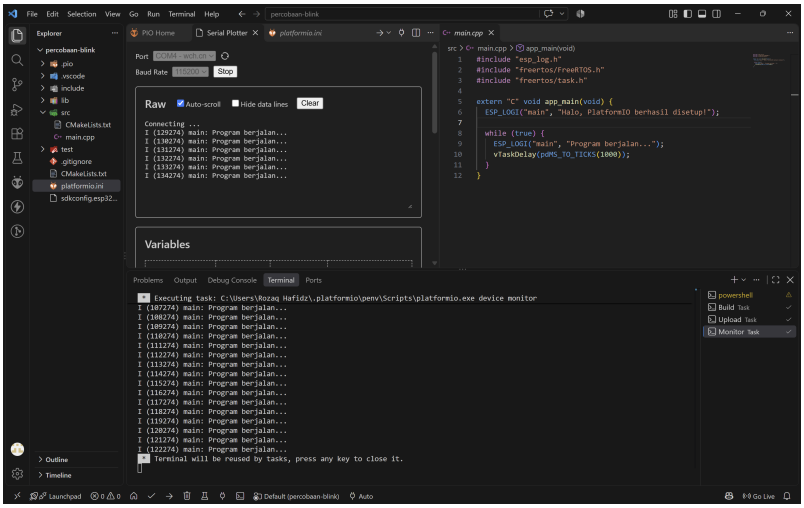
```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = espidf
```

Tunggu pemasangan paket dan pembaruan konfigurasi selesai sebelum Build. PlatformIO mengelola toolchain serta framework yang dibutuhkan. Untuk Upload, hubungkan ESP32 memakai kabel USB data dan pastikan driver chip CP210x/CH340 yang sesuai serta port serial terdeteksi.

d. Instal ekstensi *Serial Plotter* yang mendukung pembacaan port serial. Buka panel ekstensi melalui perintah yang tersedia pada Command Palette, pilih port board, dan samakan baud rate dengan firmware, misalnya 115200 bps.



Gambar 1: VS Code dan PlatformIO dengan ESP-IDF



Gambar 2: Serial Plotter